

```
"""
```

```
=====
```

```
Stock Scanner - فحص أسهم السوق الأمريكي  
Claude يحلل الأسهم بمؤشرات فنية شاملة + تلخيص بواسطة  
ثم يرسل التقرير على تلغرام
```

```
=====
```

```
"""
```

```
import os  
import time  
import requests  
import pandas as pd  
import numpy as np  
from datetime import datetime
```

```
# ===== الإعدادات (من GitHub Secrets) =====  
CLAUDE_API_KEY = os.environ.get("CLAUDE_API_KEY")  
TELEGRAM_BOT_TOKEN = os.environ.get("TELEGRAM_BOT_TOKEN")  
TELEGRAM_CHAT_ID = os.environ.get("TELEGRAM_CHAT_ID")  
TWELVE_DATA_API_KEY = os.environ.get("TWELVE_DATA_API_KEY")
```

```
CLAUDE_MODEL = "claude-sonnet-4-5"
```

```
# قائمة الأسهم المراد فحصها - عذّلها كما تحب  
WATCHLIST = [  
    "AAPL", "MSFT", "NVDA", "TSLA", "AMZN", "GOOGL", "META",  
    "AMD", "NFLX", "PLTR", "SMCI", "AVGO", "COIN", "MSTR"  
]
```

```
# ===== Twelve Data جلب البيانات من =====  
def fetch_stock_data(ticker, outputsize=250, interval="1day"):  
    if not TWELVE_DATA_API_KEY:  
        print("⚠️ Twelve Data API لم يتم ضبط مفتاح")  
        return None  
  
    url = "https://api.twelvedata.com/time_series"  
    params = {  
        "symbol": ticker,  
        "interval": interval,  
        "outputsize": outputsize,  
        "apikey": TWELVE_DATA_API_KEY,  
    }
```

```

try:
    resp = requests.get(url, params=params, timeout=20)
    data = resp.json()

    if data.get("status") == "error" or "values" not in data:
        print(f"⚠️ خطأ في جلب بيانات {ticker}: {data.get('message', data)}")
        return None

    values = data["values"]
    if len(values) < 30:
        return None

    df = pd.DataFrame(values)
    df = df.rename(columns={
        "close": "Close", "high": "High", "low": "Low",
        "open": "Open", "volume": "Volume",
    })
    for col in ["Close", "High", "Low", "Open", "Volume"]:
        df[col] = pd.to_numeric(df[col], errors="coerce")

    # Twelve Data أولاً، نعكسها عشان تصير الأقدم أولاً
    df = df.iloc[::-1].reset_index(drop=True)
    df.dropna(inplace=True)

    return df
except Exception as e:
    print(f"⚠️ خطأ في جلب بيانات {ticker}: {e}")
    return None

# ===== حساب المؤشرات الفنية الشاملة =====
def calculate_indicators(df):
    close = df["Close"]
    high = df["High"]
    low = df["Low"]
    volume = df["Volume"]

    ind = {}

    # المتوسطات المتحركة
    ind["MA20"] = close.rolling(20).mean().iloc[-1]
    ind["MA50"] = close.rolling(50).mean().iloc[-1]
    ind["MA200"] = close.rolling(200).mean().iloc[-1] if len(close) >= 200 else N

    # RSI
    delta = close.diff()
    gain = delta.where(delta > 0, 0).rolling(14).mean()

```

```

loss = -delta.where(delta < 0, 0).rolling(14).mean()
rs = gain / loss
ind["RSI"] = (100 - (100 / (1 + rs))).iloc[-1]

# MACD
ema12 = close.ewm(span=12, adjust=False).mean()
ema26 = close.ewm(span=26, adjust=False).mean()
macd_line = ema12 - ema26
signal_line = macd_line.ewm(span=9, adjust=False).mean()
ind["MACD"] = macd_line.iloc[-1]
ind["MACD_signal"] = signal_line.iloc[-1]
ind["MACD_hist"] = (macd_line - signal_line).iloc[-1]

# Bollinger Bands
ma20 = close.rolling(20).mean()
std20 = close.rolling(20).std()
ind["BB_upper"] = (ma20 + 2 * std20).iloc[-1]
ind["BB_lower"] = (ma20 - 2 * std20).iloc[-1]

# Stochastic Oscillator
low14 = low.rolling(14).min()
high14 = high.rolling(14).max()
k = 100 * (close - low14) / (high14 - low14)
ind["Stoch_K"] = k.iloc[-1]
ind["Stoch_D"] = k.rolling(3).mean().iloc[-1]

# ATR (تذبذب)
tr1 = high - low
tr2 = (high - close.shift()).abs()
tr3 = (low - close.shift()).abs()
tr = pd.concat([tr1, tr2, tr3], axis=1).max(axis=1)
ind["ATR"] = tr.rolling(14).mean().iloc[-1]

# ADX (قوة الترنند)
plus_dm = high.diff()
minus_dm = -low.diff()
plus_dm[plus_dm < 0] = 0
minus_dm[minus_dm < 0] = 0
tr14 = tr.rolling(14).sum()
plus_di = 100 * (plus_dm.rolling(14).sum() / tr14)
minus_di = 100 * (minus_dm.rolling(14).sum() / tr14)
dx = 100 * (plus_di - minus_di).abs() / (plus_di + minus_di)
ind["ADX"] = dx.rolling(14).mean().iloc[-1]

# حجم التداول
ind["Volume"] = volume.iloc[-1]
ind["Avg_Volume_20"] = volume.rolling(20).mean().iloc[-1]

```

```

ind["Volume_ratio"] = ind["Volume"] / ind["Avg_Volume_20"] if ind["Avg_Volume

# السعر والتغير
ind["Price"] = close.iloc[-1]
ind["Change_pct"] = ((close.iloc[-1] - close.iloc[-2]) / close.iloc[-2]) * 10

# الدعم والمقاومة
ind["Resistance"] = high.rolling(20).max().iloc[-1]
ind["Support"] = low.rolling(20).min().iloc[-1]

return ind

# ===== منطق اختيار الفرص الجيدة =====
def is_good_opportunity(ind):
    if ind["MA200"] is None:
        return False

    conditions = [
        ind["Price"] > ind["MA50"],          # فوق المتوسط 50
        ind["MA50"] > ind["MA200"],          # ترند صاعد عام
        30 < ind["RSI"] < 70,                # ليس متطرف
        ind["MACD_hist"] > 0,                # زخم إيجابي
        ind["Volume_ratio"] > 1.2,           # حجم أعلى من المعتاد
        ind["ADX"] > 20,                     # قوة اتجاه
    ]
    return sum(conditions) >= 4

# ===== Claude تلخيص بواسطة =====
def get_claude_summary(ticker, ind):
    if not CLAUDE_API_KEY:
        return "⚠️ Claude API لم يتم ضبط مفتاح"

    prompt = f"""حلل السهم {ticker} بالعربي (سطين إلى ثلاثة)
وأعطني ملخص قصير جداً (سطين إلى ثلاثة) بالعربي
يوضح لماذا يعتبر فرصة جيدة الآن، بدون مقدمات

السعر الحالي: {ind['Price']:.2f}
التغير اليومي: {ind['Change_pct']:.2f}%
RSI: {ind['RSI']:.1f}
MACD Histogram: {ind['MACD_hist']:.3f}
MA50: {ind['MA50']:.2f}
MA200: {ind['MA200']:.2f}
ADX: {ind['ADX']:.1f}
نسبة حجم التداول للمتوسط: {ind['Volume_ratio']:.2f}
الدعم: {ind['Support']:.2f}
المقاومة: {ind['Resistance']:.2f}"""

```

```

headers = {
    "x-api-key": CLAUDE_API_KEY,
    "anthropic-version": "2023-06-01",
    "content-type": "application/json",
}
body = {
    "model": CLAUDE_MODEL,
    "max_tokens": 300,
    "messages": [{"role": "user", "content": prompt}],
}

try:
    resp = requests.post(
        "https://api.anthropic.com/v1/messages",
        headers=headers,
        json=body,
        timeout=30,
    )
    data = resp.json()
    if "content" in data:
        texts = [b["text"] for b in data["content"] if b.get("type") == "text"]
        return "\n".join(texts).strip()
    return f"Claude: خطأ من {data}"
except Exception as e:
    return f"Claude: خطأ بالاتصال مع {e}"

# ===== تشغيل الفحص الكامل =====
def run_scan():
    results = []
    for ticker in WATCHLIST:
        df = fetch_stock_data(ticker)
        time.sleep(8) # (8 طلبات/دقيقة) المجانية
        if df is None:
            continue

        ind = calculate_indicators(df)

        if any(pd.isna(v) for v in ind.values() if isinstance(v, (int, float))):
            continue

        if is_good_opportunity(ind):
            summary = get_claude_summary(ticker, ind)
            results.append({"ticker": ticker, "indicators": ind, "summary": summary})

    return results

```

```

# ===== بناء رسالة تليغرام =====
def build_telegram_message(results):
    now = datetime.now().strftime("%Y-%m-%d %H:%M")

    if not results:
        return f"🇲🇦 *فحص أسهم - {now}*\nالفحص حسب معايير الفحص الحالية لا توجد فرص قوية حالياً حسب معايير الفحص."

    msg = f"📈 *فحص أسهم - {now}*\n"
    msg += f"عدد الفرص: {len(results)}\n\n"

    for r in results:
        t = r["ticker"]
        ind = r["indicators"]
        msg += "—————\n"
        msg += f"💎 *{t}*\n"
        msg += f"السعر: {ind['Price']:.2f} ({ind['Change_pct']:+.2f}%)\n"
        msg += f"RSI: {ind['RSI']:.1f} | MACD: {'إيجابي' if ind['MACD_hist']>0 else 'سلبي'}\n"
        msg += f"MA50: {ind['MA50']:.2f} | MA200: {ind['MA200']:.2f}\n"
        msg += f"ADX: {ind['ADX']:.1f} | نسبة الحجم: {ind['Volume_ratio']:.2f}x\n"
        msg += f"الدعم: {ind['Support']:.2f} | المقاومة: {ind['Resistance']:.2f}\n"
        msg += f"📄 {r['summary']}\n\n"

    return msg

# ===== إرسال تليغرام =====
def send_telegram_message(text):
    if not TELEGRAM_BOT_TOKEN or not TELEGRAM_CHAT_ID:
        print("⚠️ لم يتم ضبط بيانات تليغرام (TELEGRAM_BOT_TOKEN / TELEGRAM_CHAT_ID)")
        return

    url = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage"

    max_len = 4000
    chunks = [text[i:i + max_len] for i in range(0, len(text), max_len)]

    for chunk in chunks:
        payload = {
            "chat_id": TELEGRAM_CHAT_ID,
            "text": chunk,
            "parse_mode": "Markdown",
        }
        try:
            resp = requests.post(url, json=payload, timeout=15)
            if resp.status_code != 200:

```

```

        print(f"⚠️ خطأ إرسال تلقرام: {resp.text}")
    else:
        print(f"✅ تم إرسال جزء من الرسالة")
except Exception as e:
    print(f"⚠️ خطأ بالاتصال مع تلقرام: {e}")

# ===== نقطة التشغيل =====
if __name__ == "__main__":
    print("🔍 بدء فحص الأسهم")
    scan_results = run_scan()
    print(f"✅ فرصة {len(scan_results)} تم العثور على")

    message = build_telegram_message(scan_results)
    send_telegram_message(message)
    print("🎉 انتهى التقرير")

```